Security overview document on Secure File Sharing System

This document provides detailed information on the encryption and key handling methods utilised in developing the secure file sharing system.

1. Technical Architecture:

The system uses a Client–Server model, with the frontend (index.html) capturing user input and the backend (app.py) handling the cryptographic processing through the PyCryptodome library, ensuring that all files are encrypted before being stored or transmitted.
This architectural separation also ensures that operations remain fully protected and never exposed to the user or the client-side environment.

```python
import os
import io
from flask import Flask, request, send_file, render_template
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad

app = Flask(__name__)

# ── CONFIGURATION ──
# This ensures the vault folder is always found inside your project folder
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
UPLOAD_FOLDER = os.path.join(BASE_DIR, 'vault')
SECRET_KEY = b'SixteenByteKey!!' # Must be exactly 16, 24, or 32 bytes

# Create the vault folder if it doesn't exist
if not os.path.exists(UPLOAD_FOLDER):
    os.makedirs(UPLOAD_FOLDER)

@app.route('/')
def index():
    return render_template('index.html')
```

Figure 1a: Backend architectural configuration using Flask and PyCryptodome for secure cryptographic services.

2. Encryption Method:

This section outlines the technique used to protect data by converting readable information into a secure, unreadable format.

2.1. AES-256 encryption in Cipher Block Chaining (CBC) mode: I implemented AES-256, a highly secure variant of the Advanced Encryption Standard that uses a 256-bit key. Its robust and complex encryption structure makes it extremely resistant to brute-force attempts. As a symmetric algorithm, it relies on the same key for both encryption and decryption, ensuring efficient and seamless data protection.

Additionally, the **CBC** mode was implemented because it offers stronger security than many other block cipher modes. By linking each plaintext block to the ciphertext block before it, CBC makes the resulting encrypted output far less predictable. It also uses a fresh, random Initialization Vector (IV) for every encryption operation, ensuring that identical inputs generate

different ciphertexts. This prevents pattern recognition and significantly reduces the risk of attackers exploiting repeated data.

```
35      # 2. Setup AES Encryption (CBC Mode)
36      cipher = AES.new(SECRET_KEY, AES.MODE_CBC)
```

Figure 2a: Initializing the AES cipher object in CBC mode using the defined SECRET_KEY.

2.2. Data Padding: Since AES operates on fixed 16-byte blocks, PKCS7 padding is applied to ensure that any input—regardless of its original length—fits the required block size. This padding scheme not only guarantees smooth encryption and decryption but also prevents data truncation errors, maintains structural consistency across all encrypted messages, and ensures compatibility with modes like CBC that strictly require complete blocks. Using PKCS7 also standardizes the padding process, making it predictable, reversible, and widely supported across cryptographic libraries.

```
38      # 3. Encrypt the data with padding
39      ct_bytes = cipher.encrypt(pad(data, AES.block_size))
40
```

Figure 2b: Applying PKCS7 padding to the original data before executing the encryption command.

2.3. The process: Within the upload_file() function, the system reads the incoming data, initializes a new cipher object, and then performs the encryption operation to produce the ciphertext.

```
24 def upload_file():
25      if 'file' not in request.files:
26          return "No file part", 400
```

Figure 2c: Configuration for process implementation.

3. Key Handling Strategy

The system handles security keys and session parameters through a structured, controlled process that ensures confidentiality and integrity at every stage. It securely generates, stores, and applies cryptographic keys while also managing session-specific values, such as IVs and padding requirements, to maintain consistent, tamper-resistant encryption operations.

The strategy is outlined in more detail below:

3.1. Secret Key Definition: The system relies on a fixed 16-byte key () declared at the top of the script. This key serves as the core cryptographic material used during both encryption and decryption, ensuring that data can be securely encrypted and later decrypted using the same consistent key.

```
12 UPLOAD_FOLDER = os.path.join(BASE_DIR, 'vault')
13 SECRET_KEY = b'SixteenByteKey!!' # Must be exactly 16, 24, or 32 bytes
```

Figure 3a: Definition of Secret key.

3.2. Automatic IV Generation: During Step 2 of the upload process, the system initializes the AES cipher in CBC mode. Since no IV is manually supplied in the AES.new() call, the library automatically produces a secure, random Initialization Vector for that specific encryption operation. This ensures that each file receives a unique IV, strengthening security by preventing attackers from correlating patterns across multiple encrypted inputs.

```
35      # 2. Setup AES Encryption (CBC Mode)
36      cipher = AES.new(SECRET_KEY, AES.MODE_CBC)
```

Figure 3b: Implementation of the automated IV generation.

3.3 Secure Storage (The Vault): To support reliable decryption later, the system stores both the IV and the ciphertext together in a single output file. The IV occupies the first 16 bytes, followed by the encrypted data, and this combined file is then saved inside the vault directory. This structure ensures that all necessary components for decryption are preserved in a single secure location, thereby reducing the risk of key material loss or mismatched parameters.

```
41      # 4. Save the IV + Ciphertext together in the vault
42      file_path = os.path.join(UPLOAD_FOLDER, f"{file.filename}.enc")
43      with open(file_path, "wb") as f:
44          f.write(cipher.iv)  # Write the 16-byte IV first
45          f.write(ct_bytes)   # Then write the encrypted data
46
```

Figure 3c: Unified storage of the IV and ciphertext (.enc) file format.

3.4. Key Recovery during Decryption: When a user downloads a file, the system extracts the first 16 bytes to retrieve the stored IV. This IV is then combined with the SECRET_KEY to re-initialize the AES cipher, allowing the encrypted data that follows to be correctly decrypted and restored to its original form.

```
58          # 2. Read the first 16 bytes to get the IV
59          iv = f.read(16)
60          encrypted_data = f.read()
61
62      # 3. Setup Decryption
63      cipher = AES.new(SECRET_KEY, AES.MODE_CBC, iv)
64
```

Figure 3d: Retrieving the stored 16-byte IV and encrypted data for decryption.

4. Data Integrity and Storage:

The encrypted output is stored in binary format (wb) within the vault to ensure maximum storage efficiency. This binary data can be represented in Base64 or Hexadecimal to preserve data integrity when the file is transmitted over the network or stored in a database, ensuring the content remains intact and fully recoverable during decryption.

```
43     with open(file_path, "wb") as f:
44         f.write(cipher.iv)  # Write the 16-byte IV first
45         f.write(ct_bytes)   # Then write the encrypted data
46
```

Figure 4a: Binary storage logic in app.py.

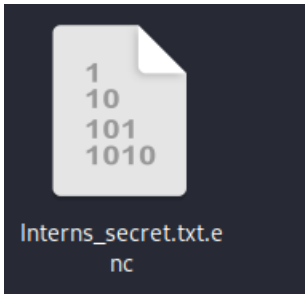

Figure 4b: Encoded ciphertext output.

5. Security Measures Implemented

The following security best practices were embedded directly into the system's architecture:

5.1. Automated Vault Creation: The system automatically verifies whether the vault directory exists and creates it when necessary. This prevents sensitive files from being stored in the project root, ensuring that all encrypted data is organised, contained, and kept within a designated secure location.

```
15 # Create the vault folder if it doesn't exist
16 if not os.path.exists(UPLOAD_FOLDER):
17     os.makedirs(UPLOAD_FOLDER)
18
```

Figure 5a: Backend configuration for automated vault folder creation and secure file pathing.

5.2. Error Handling: The system uses a try–except block during the decryption process to detect and manage errors such as "incorrect padding" or "invalid key." By handling these exceptions, the application avoids crashing and prevents sensitive system information from being exposed—reducing the risk of giving attackers insight into the system's internal workings.

```
65     # 4. Decrypt and strip padding
66     try:
67         decrypted_data = unpad(cipher.decrypt(encrypted_data), AES.block_size)
68     except ValueError:
69         return "Decryption failed: Incorrect padding or key.", 500
70
```

Figure 5b: Defensive programming to handle decryption failures.

5.3. Streamlined Data Flow: By leveraging io.BytesIO, the system handles decrypted files entirely in memory before delivering them to the browser. This approach minimizes the creation of temporary unencrypted files on the server's hard drive, thereby reducing exposure risks and improving overall performance by keeping sensitive data out of persistent storage.

```
    # 5. Send decrypted file back to browser
    return send_file(
        io.BytesIO(decrypted_data),
        download_name=filename,
        as_attachment=True
    )
```

Figure 5c: Implementation of memory-resident data handling to minimize persistent storage exposure.

6. Conclusion

The data protection system delivers a secure, locally hosted "Vault" powered by industry-standard AES-256 encryption in CBC mode. With automated IV generation and a clear separation between the encryption engine and the user interface, the application maintains strong confidentiality and integrity for all protected files. Overall, the project showcases a reliable and well-structured approach to secure file management.